1. 从增强已有的css属性开始

css这门语言入门很简单，即使一个完全没有接触过的人也能很快上手，因为它的常用属性就那么多，使用方式也很直观，无非就是选择器加属性；这就导致很多人没有系统的学习过，属于会用但不知道为什么，在碰到一些怪异现象的时候就会摸不着头脑；

本课程将基于张鑫旭的《css新世界》，系统的学习一下css3；这本书的主要内容在于对css3新属性的讲解，笔者将会按照原书的内容结构阅读并总结，筛选出一些实用性较强的属性或原理性知识，并加入一些自己的理解进行分享

1.1 Css3的历史背景

在css2时代，互联网刚刚兴起，硬件设备还很差，网页主要用于简单的图文展示，因此css2的设计也并没有考虑复杂的布局效果等；但随着ajax的兴起，网页的交互和布局都越来越复杂，css2已无法满足需求，因此有了css3；

相比于css2，css3有以下特点：

1. 更复杂的布局场景和效果支持，如flex，grid等
2. 语法设计明确，专项专用
3. 属性值数量大增，特性间的相互干涉减少

值得一提的是，不会再有Css4了！因为Css3已经解体了，因为Css按照大版本迭代的方式效率太低了；CSS2.1的规范在2002年开始制定，而一直到 2011年才成为推荐规范，整个过程足足花费了10年的时间，就是因为 一些次要的CSS特性拖了后腿。为了快速响应时代的变化，W3C将Css分成了各个模块小组，各模块各自迭代发布；例如，CSS选择器模块已经到了Level 4了，弹性布局模块还是Level 1；

在学习css2中我们经常会遇到一些奇奇怪怪的问题，比如float导致的父元素高度塌陷，这是因为float属性设计的初衷本来就不是用于模块布局，而是用于文字环绕效果；在css3中就尽量避免了这种情况的出现，布局的属性就用于布局，视觉展示的属性就用于视觉展示，并且没有特性间的相互干涉；理解成本的降低导致了记忆成本的提高，css3新特性的属性值非常多，为了提高学习的效率我们有必要先学会看懂css属性值的定义语法；

1.2 读懂css属性值定义语法

css的值定义语法主要由三部分构成，关键字，符号和数据类型

关键字 数据类型

1.2.1 关键字

关键字分为一般关键字和全局关键字

一般关键字只可以在部分特定的属性中使用，如auto，none，ease等；而全局关键字则是被所有css属性支持的，有：inherit，initial，revert 和 unset；其中比较实用的有inherit和initial；

`inherit` 就是继承的意思，子元素可以设置 `height:inherit` 实现高度继承，或者子元素 设置 `background-image:inherit` 实现背景图像继承等；比较常见的场景是用它来重置输入框内的默认字体

CSS

```
input, textarea {
    font-family: inherit;
}
```

`initial` 用于恢复css属性的初始值， `revert` 用于还原某元素属性的浏览器预设值；注意不要将二者弄混， `initial` 针对的是css属性本身，而 `revert` 针对的是元素的属性

CSS

```
div {
    display: initial // inline
}
div {
    display: revert // block
}
```

`unset` 对于具有继承特性的css属性等同于使用 `inherit` 关键字 ，对于没有继承特性的css属性等同于使用 `initial` 关键字；没有太大实用价值；

1.2.2 数据类型

像<image>，<color-stop-list>这样由尖括号包裹的一个单元就是一个数据类型，一个数据类型可以内含其它的数据类型，关键字或函数；前面我们说到css3的属性值数量大增，如果我们还是按照

使用经验来记忆的话效率就会比较低；我们可以看看<image>数据类型的值感受一下

```
<bg-image>#
where
<bg-image> = none | <image>
where
<image> = <url> | <image()> | <image-set()> | <element()> | <paint()> | <cross-fade()> |
<gradient>
where
<image()> = image( <image-tags>? [ <image-src>? , <color>? ]! )
<image-set()> = image-set( <image-set-option># )
<element()> = element( <id-selector> )
<paint()> = paint( <ident> (en-US), <declaration-value>? )
<cross-fade()> = cross-fade( <cf-mixing-image> , <cf-final-image>? )
<gradient> = <linear-gradient()> | <repeating-linear-gradient()> | <radial-gradient()> |
<repeating-radial-gradient()> | <conic-gradient()>
where
<image-tags> = ltr | rtl
<image-src> = <url> | <string>
<color> = <rgb()> | <rgba()> | <hsl()> | <hsla()> | <hwb()> | <hex-color> | <named-color> |
currentcolor | <deprecated-system-color>
<image-set-option> = [ <image> | <string> ] [ <resolution> || type(<string>) ]
<id-selector> = <hash-token>
<cf-mixing-image> = <percentage>? && <image>
<cf-final-image> = <image> | <color>
```

数据类型的好处在于，当我们看到两个属性值的数据类型一致的时候，我们就能知道这两个属性的语法是一致的比如最简单常见的：width的值，<length> 数据类型，由一个<number>和一个长度单位组成，涉及尺寸的一般都能用；

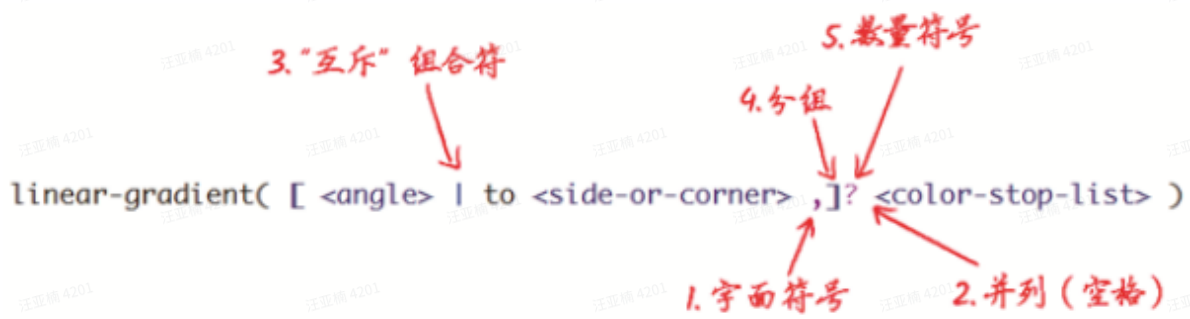
1.2.3 符号

css语法中的符号有三种，第一种是字面符号，有逗号和斜杠；`,`逗号用来分隔数个并列值，或者分隔函数的参数；`/` 斜杠用来分隔一个值的多个部分，通常用在CSS缩写中分离具有相同数据类型但属于不同属性的值；还有两种分别是组合符号和数量符号，大部分用法和编程语言或正则中差不多，具体可以看 MDN

符号	名称	描述	示例
组合符号			
	并置	各部分必须出现且按顺序出现	`solid <length>`
`&&`	“与”组合符	各部分必须出现，但可以不按顺序	`<length> && <string>`

`||`	“或”组合符	各部分至少出现一个，可以不按顺序	`<'border-image-outset'> || <'border-image-slice'>`
`|`	“互斥”组合符	各部分恰好出现一个	`smaller | small | normal | big | bigger`
`[ ]`	方括号	强调优先级	`bold [ thin && <length> ]`
数量符号			
	无数量符号	恰好一次	`solid`
`*`	星号	零次、一次或多次	`bold smaller*`
`+`	加号	一次或多次	`bold smaller+`
`?`	问号	零次或一次（即可选）	`bold smaller?`
`{A,B}`	大括号	至少 `A` 次，至多 `B` 次	`bold smaller{1,3}`
`#`	井号	一次或多次，但多次出现必须以逗号分隔	`bold smaller#`
`!`	叹号	组必须产生一个值	`[ bold? smaller? ]!`

学会了css的语法定义，我们就可以通过阅读官方文档来使用一些相对复杂的属性了



1.3 css布局相关概念

1.3.1 视觉格式化模型

这一节并不是书本中的内容，但与布局或尺寸相关的概念都有所关联，我们在这里简单回顾一下；

提到视觉格式化模型，涉及的块啊盒的概念非常多；

译注：本文有很多相近的术语，阅读时需仔细，否则容易造成误解。为了方便读者，这里我将其整理一下。

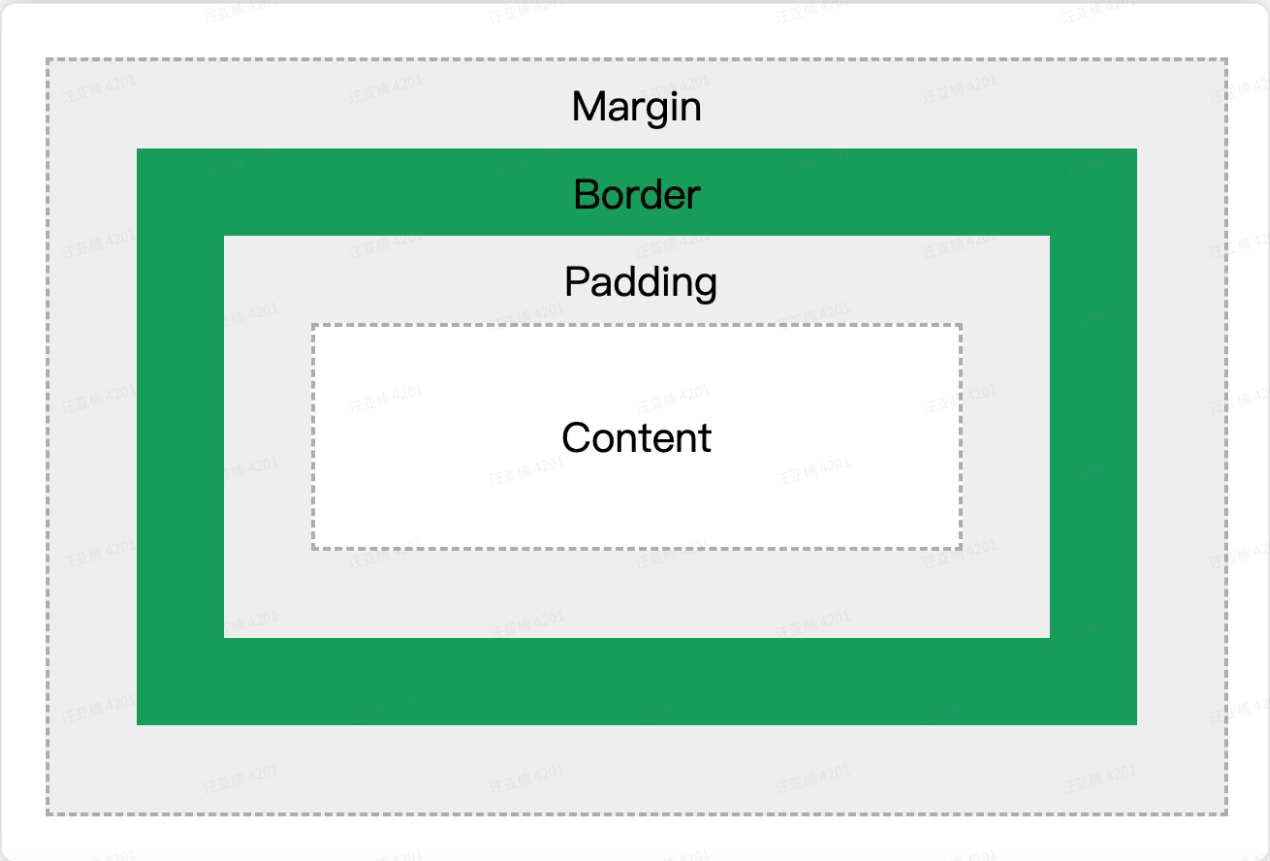
- **块**：block，一个抽象的概念，一个块在文档流上占据一个独立的区域，块与块之间在垂直方向上按照顺序依次堆叠。
- **包含块**：containing block，包含其他盒子的块称为包含块。
- **盒子**：box，一个抽象的概念，由CSS引擎根据文档中的内容所创建，主要用于文档元素的定位、布局和格式化等用途。盒子与元素并不是一一对应的，有时多个元素会合并生成一个盒子，有时一个元素会生成多个盒子（如匿名盒子）。
- **块级元素**：block-level element，元素的 `display` 为 `block`、`list-item`、`table` 时，该元素将成为块级元素。元素是否是块级元素仅是元素本身的属性，并不直接用于格式化上下文的创建或布局。
- **块级盒子**：block-level box，由块级元素生成。一个块级元素至少会生成一个块级盒子，但也有可能生成多个（例如列表项元素）。
- **块盒子**：block box，如果一个块级盒子同时也是一个块容器盒子（见下），则称其为块盒子。除具名块盒子之外，还有一类块盒子是匿名的，称为匿名块盒子（Anonymous block box），匿名盒子无法被CSS选择符选中。
- **块容器盒子**：block container box或block containing box，块容器盒子侧重于当前盒子作为“容器”的这一角色，它不参与当前块的布局和定位，它所描述的仅仅是当前盒子与其后代之间的关系。换句话说，块容器盒子主要用于确定其子元素的定位、布局等。

注意：盒子分为“块盒子”和“块级盒子”两种，但元素只有“块级元素”，而没有“块元素”。下面的“行内级元素”也是一样。

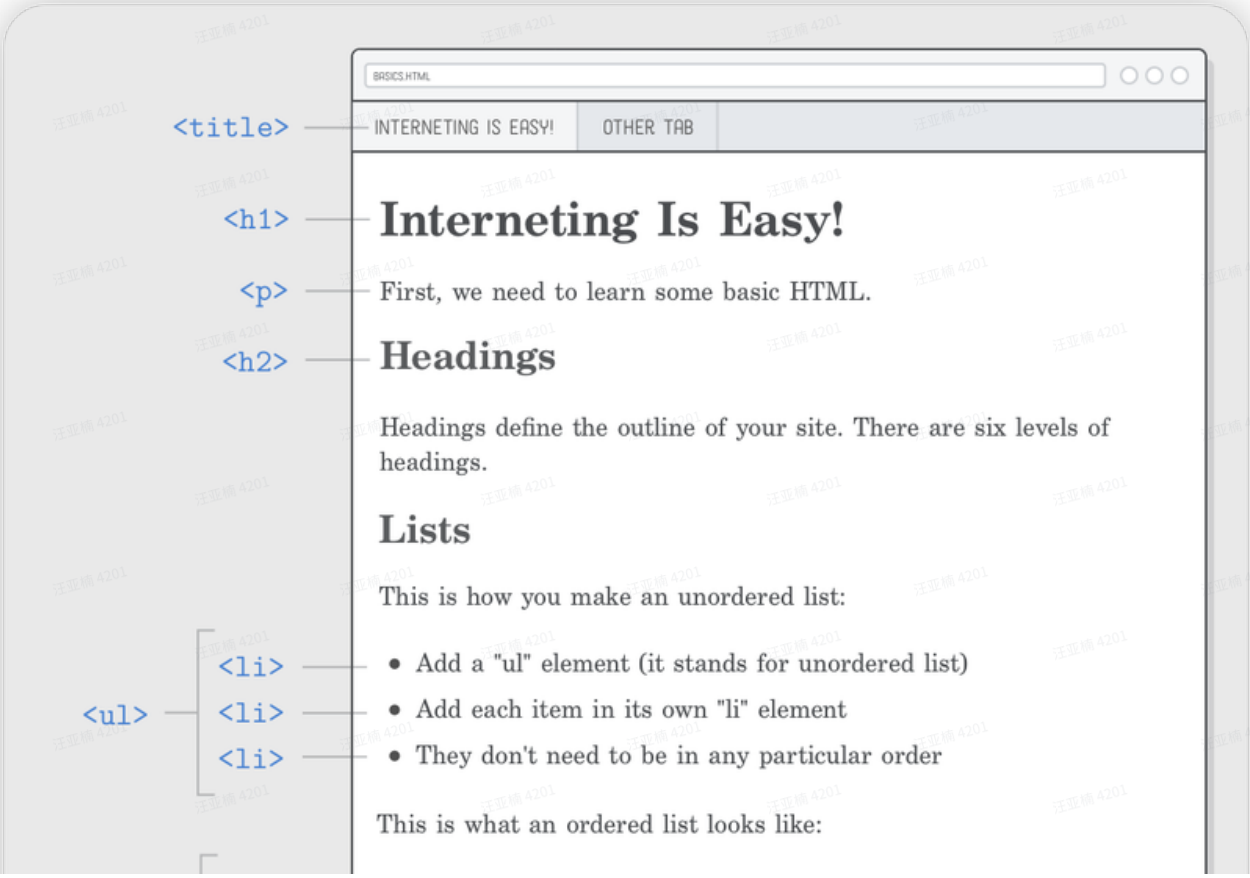
- **行内级元素**：inline-level element，`display` 为 `inline`、`inline-block`、`inline-table` 的元素称为行内级元素。与块级元素一样，元素是否是行内级元素仅是元素本身的属性，并不直接用于格式化上下文的创建或布局。
- **行内级盒子**：inline-level box，由行内级元素生成。行内级盒子包括行内盒子和原子行内级盒子两种，区别在于该盒子是否参与行内格式化上下文的创建。
- **行内盒子**：inline box，参与行内格式化上下文创建的行内级盒子称为行内盒子。与块盒子类似，行内盒子也分为具名行内盒子和匿名行内盒子（anonymous inline box）两种。
- **原子行内级盒子**：atomic inline-level box，不参与行内格式化上下文创建的行内级盒子。原子行内级盒子一开始叫做原子行内盒子（atomic inline box），后被修正。原子行内级盒子的内容不会拆分成多行显示。

另外，本文的英文原文仍未最后定稿，因此部分内容并不完整。待英文原文更新后应及时更新译文。

我们知道，页面上的任何元素都是一个个盒子，而这些盒子的抽象概念即 **盒模型**，它决定和盒子如何渲染；



在默认情况下元素按自左而右，自上而下的规则排列，即**文档流；**



<ol>
<li> 1. Notice the new "ol" element wrapping everything
<li> 2. But, the list item elements are the same
<li> 3. Also note how the numbers increment on their own
<li> 4. You should be noticing things in this precise order, because this is an ordered list

Inline Elements

 Sometimes, you need to draw attention to a particular word or phrase.

Other times, you need to **strong**ly emphasize the importance of a word or phrase.

  And sometimes you need to shout!

Empty Elements

Thanks for reading! Interneting should be getting easier now.

Regards,
The Authors

<hr/>

P.S. This page might look like crap, but we'll learn how to fix that with some CSS soon.

在文档流有些元素会自占一行，上下堆叠，有些元素则是水平排列；我们知道这是因为有的元素是块元素，有的元素行内元素；为什么不同的元素类型会导致这些不同呢？

这是因为这些元素处于或者产生了有不同渲染规则的渲染区域，这些渲染区域的概念就是**格式化上下文**；

提到格式化上下文，我们最熟悉的应该就是**BFC（Block Formatting Context）块级格式化上下文**了；

> **块格式化上下文（Block Formatting Context，BFC）** 是Web页面的可视CSS渲染的一部分，是块盒子的布局过程发生的区域，也是浮动元素与其他元素交互的区域
>
> 下列方式会创建**块格式化上下文**：
>
> - 根元素（`<html>`）
> - 浮动元素（元素的 `float` 不是 `none`）
> - 绝对定位元素（元素的 `position` 为 `absolute` 或 `fixed`）
> - 行内块元素（元素的 `display` 为 `inline-block`）
> - 表格单元格（元素的 `display` 为 `table-cell`，HTML表格单元格默认为该值）
> - 表格标题（元素的 `display` 为 `table-caption`，HTML表格标题默认为该值）
> - 匿名表格单元格元素（元素的 `display` 为 `table`、`table-row`、`table-row-group`、`table-header-group`、`table-footer-group`（分别是HTML table、row、tbody、thead、tfoot 的默认属性）或 `inline-table`）
> - `overflow` 计算值(Computed)不为 `visible` 的块元素
> - `display` 值为 `flow-root` 的元素
> - `contain` 值为 `layout`、`content` 或 paint 的元素
> - 弹性元素（`display` 为 `flex` 或 `inline-flex` 元素的直接子元素）
> - 网格元素（`display` 为 `grid` 或 `inline-grid` 元素的直接子元素）
> - 多列容器（元素的 `column-count` 或 `column-width` (en-US) 不为 `auto`，包括 `column-count` 为 `1`）
> - `column-span` 为 `all` 的元素始终会创建一个新的BFC，即使该元素没有包裹在一个多列容器中（标准变更，Chrome bug）

块级格式化上下文有以下约束：

> - BFC内部的BOX会在垂直方向上一个接一个排列
> - 垂直方向上测距离由margin决定，属于同一个BFC的两个相邻box的margin会发生重叠。
> - 每个元素的左外边距与包含块的左边界相接触（从左向右），即使浮动元素也是如此（这说明BFC中的子元素不会超出他的包含块，而position为absolute的元素可以超出他的包含块边界）
> - BFC的区域不会与float的元素区域重叠。
> - 计算BFC高度时,浮动子元素也参与计算。
> - BFC就是页面上的一个隔离的独立容器，容器里的子元素不会影响到外面的元素，反之亦然

从上面BFC的创建条件中可以看到根元素就会创建一个BFC，因此整个网页其实就是在一个BFC中的，BFC的约束条件第一条就是盒在BFC中会以垂直方向排列，所以我们的元素会自上而下排列；

那为什么行内元素会被水平排列呢？

这是因为这些元素处在**IFC（Inline Formatting Context）行内格式化上下文**中；在行内格式化上下文的约束有一条就是元素按水平布局，因此行内级元素都是水平排列的！

在已发布的规范文档中，笔者并没有找到明确的IFC的创建条件，但是在草案中我们可以看到：

> 9.4.2 Inline formatting contexts
>
> An inline formatting context is established by a block container box that contains no block-level boxes. In an inline formatting context, boxes are laid out horizontally, one after the other, beginning at the top of a containing block. Horizontal margins, borders, and padding are respected between these boxes. The boxes may be aligned vertically in different ways: their bottoms or tops may be aligned, or the baselines of text within them may be aligned. The rectangular area that contains the boxes that form a line is called a ***line box***.

当一个块容器内部不存在块级盒子时，它就会创建一个行内格式化上下文；所以，一般情况下，我们直接在div中书写的文本，span等都会处在行内格式化上下文当中；

那如果我们在一个块级盒子中即直接写文本，又加一些块级元素会发生什么呢？

https://codepen.io/dqqbl/pen/abEBWWP

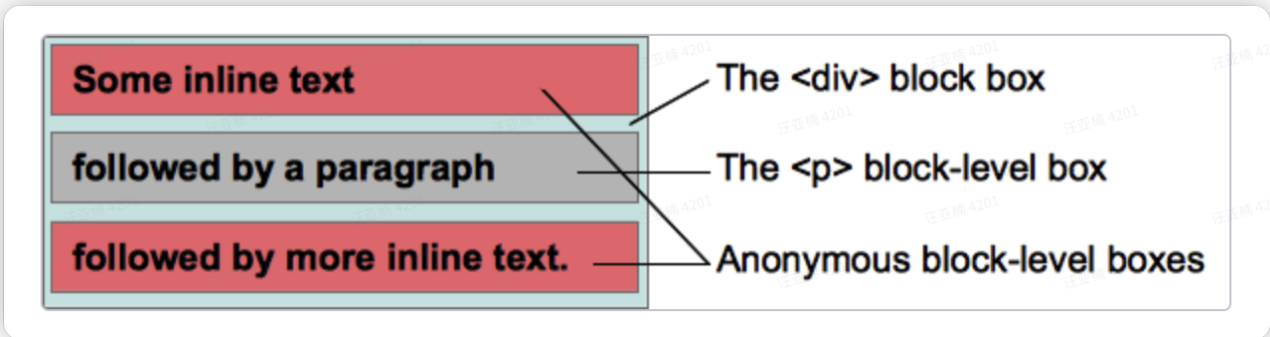
可以看到原本应该一行排布的文本被p标签分割开来，各自独占一行了

这时就引入了一个新的概念 **匿名盒子；**

> 在某些情况下进行视觉格式化时，需要添加一些增补性的盒子，这些盒子不能用CSS选择符选中，因此称为匿名盒子（*anonymous boxes*）。
>
> CSS选择器不能作用于匿名盒子(*anonymous boxes*)，所以它不能被样式表赋予样式。也就是说，此时所有可继承的 CSS 属性值都为 `inherit` ，而所有不可继承的 CSS 属性值都为 `initial`。

匿名盒子也分匿名行盒子和匿名块盒子；

这里被p标签分割的文本节点就会创建匿名块盒子，我们可以打开开发者工具看一下，这些盒子是没法被选择器直接选中的，由于匿名块盒子的存在，因此p标签相邻的文本也会独占一行；



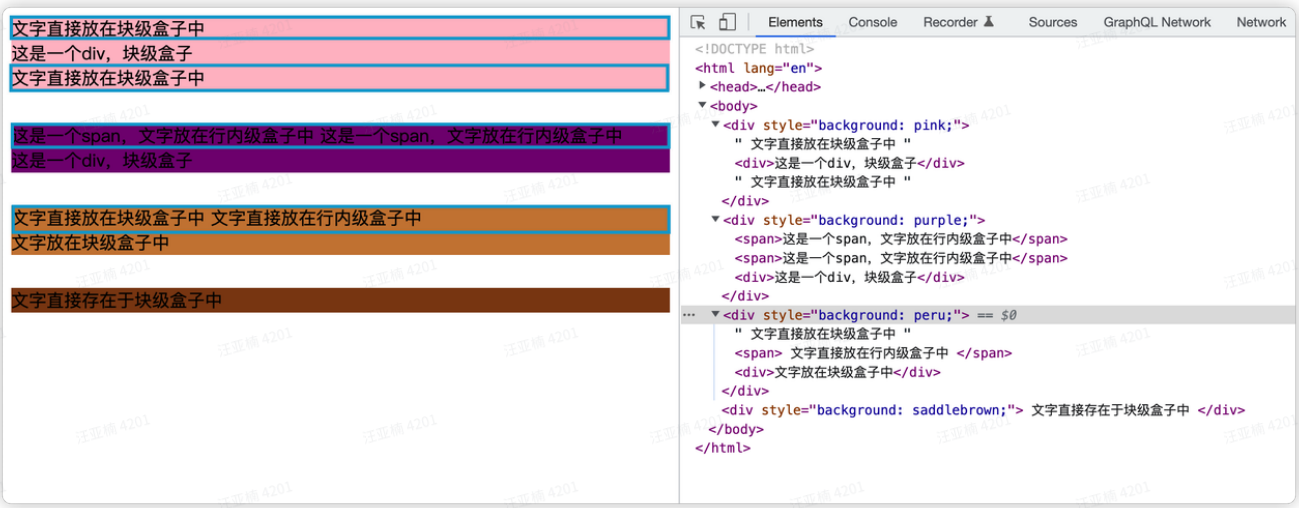
上面我们提到IFC的创建条件是，一个块容器内部不存在块级盒子，这里的p元素显然是个块级盒子，因此原先一个整体的IFC会被破坏，但在两个匿名块盒子中不存在块级盒子，因此它们会产生新的IFC；

匿名盒子的创建条件

在块包含盒子中，如果即存在块级盒子又存在行内级盒子就会在块级盒子相邻的行内级盒子外创建匿名块盒子；换而言之，只要块包含盒子内存在块级盒子，则这个块包含盒子下只存在块级盒子(所有没被文档定义的块级盒子包裹的内容都会被**匿名块盒子**包裹)

直接包含在块盒子中的文本会被一个**匿名行内盒子**包裹

参考以下面的例子，依据上面的规则，图片中被蓝线包裹的部分都创建了匿名块盒子，直接放在块级盒子中的文字则创建了匿名行内盒子



匿名盒子的存在是为了更方便我们定义文档的格式，相关内容可以看 这里

除了BFC，IFC，还有FFC（Flexbox Formatting Context，display: flex的时候创建）和GFC（Grid Formatting Contexgt，display: grid的时候创建）等；这些格式化上下文它决定了子元素如何布局，以及元素间的相互作用；而这些格式化上下文规则的集合就是 **视觉格式化模型**；

> CSS 视觉格式化模型（*visual formatting model*）是用来处理和在视觉媒体上显示文档时使用的计算规则。该模型是 CSS 的基础概念之一。
>
> 视觉格式化模型会根据CSS盒子模型将文档中的元素转换为一个个盒子，每个盒子的布局由以下因素决定：

> - 盒子的尺寸：精确指定、由约束条件指定或没有指定
> - 盒子的类型：行内盒子（inline）、行内级盒子（inline-level）、原子行内级盒子（atomic inline-level）、块盒子（block）
> - 定位方案（positioning scheme）：普通流定位、浮动定位或绝对定位
> - 文档树中的其它元素：即当前盒子的子元素或兄弟元素
> - 视口尺寸与位置
> - 所包含的图片的尺寸
> - 其他的某些外部因素

可以看到视觉格式化模型的作用也是根据将元素转化为盒子；它和盒模型之间的区别在于盒模型更专注的是盒子本身的尺寸 width，height，padding等；而视觉格式化模型关注的是盒子在页面上如何布局；

1.3.2 display属性详解

通过上面的学习我们知道，display属性可以改变元素在文档流中的表现，内在的原因是display的值改变影响了当前盒子的上下文环境，这种表现有时会让人觉得摸不着头脑；这是因为我们平时使用的display其实类似于一种缩写；一个display的值其实定义了一个盒子内外的两种表现：

- **外部显示类型(*outer display type*)**，它指示当前盒子主体本身如何参与流布局
- **内部显示类型(*inner display type*)**，它定义（如果它是一个不可替换的元素）它生成的格式化上下文的类型，指定其后代的布局方式

我们可以先来看一下规范文档：

Short 'display'	Full 'display'	Generated box
'none'	—	subtree omitted from box tree
'contents'	—	element replaced by contents in box tree
'block'	'block flow'	block-level block container aka block box
'flow-root'	'block flow-root'	block-level block container that establishes a new block formatting context (BFC)
'inline'	'inline flow'	inline box
'inline-block'	'inline flow-root'	inline-level block container aka *inline block*
'run-in'	'run-in flow'	run-in box (inline box with special box-tree-munging rules)
'list-item'	'block flow list-item'	block box with additional marker box
'inline list-item'	'inline flow list-item'	inline box with additional marker box
'flex'	'block flex'	block-level flex container

'inline-flex'	'inline flex'	inline-level flex container
'grid'	'block grid'	block-level grid container
'inline-grid'	'inline grid'	inline-level grid container
'ruby'	'inline ruby'	inline-level ruby container
'block ruby'	'block ruby'	block box containing ruby container
'table'	'block table'	block-level table wrapper box containing table grid box
'inline-table'	'inline table'	inline-level table wrapper box containing table grid box
<display-internal> types	—	layout-specific internal box

外部显示类型只有两种，block和inline，定义了它们在文档流中的布局表现；而内部显示类型影响的是当前元素的后代元素的布局，即影响了当前元素内的格式化上下文；

当内部显示类型为**flow**的时候，表示当前元素内部使用普通的流布局；其 **子元素** 就会依据 **自身的外部显示类型** 进行布局

当内部显示类型为**flow-root**的时候，其内部会创建一个新的BFC，相当于内部开启了独立的文档流；这样我们就能知道，为什么display为line-block的时候，元素即有inline的特性，又有block的特性了，因为它的外部显示类型是inline，对外表现为内联特性，可以不独占一行，而内部显示类型为flow-root，flow-root会在当前元素创建一个块级容器，因此其本身也拥有了block的特性；

1.4 新增尺寸关键字

了解了这些基础概念我们再来看css的尺寸体系；css的尺寸分为内在尺寸和外在尺寸，内在尺寸表现由元素内容决定，外在尺寸由元素的上下文决定；可以简单的理解为内在尺寸即实际内容的尺寸，外在尺寸即盒模型边界尺寸；内在尺寸可能通过“溢出”的方式大于外在尺寸；

我们知道元素尺寸相关的属性width和height的默认值都为auto，如果我们没有指定一个值，则元素会根据自身属性的不同产生不同的尺寸计算效果；在CSS2.1中，CSS中的尺寸概念都隐藏在具体的CSS用法中。例如，display:inline-block、float:left和 position:absolute等CSS声明带来的“shrink-to-fit”收缩； white-space: nowrap带来的“最大内容宽度”；连续英文字符的宽度溢出其实是因为“最小内容宽度”等；这些模糊的的概念在css3中都有了具体的定义，css3新增了四个尺寸关键字，其中 `fit-content` ，`min-content` 和 `max-content` 是“内在尺寸”，`fill-available` 是“外在尺寸”。由于新的尺寸关键字在width属性中最常用，兼容性也最好，这里以width作为示例；

fit-content

这个关键字的效果与 `display: inline-block` 等的效果类似，会有一种内联尺寸收缩的效果；但是用它的话有两个好处，一个是它保留了display的原始值，二是它会使元素有确定的尺寸值，这是非常有用的；

对于已知尺寸的元素，我们常用的居中方式有 absolute + 负外边距，margin: auto，未知尺寸的有flex，absolute + transform， inline-block + text-align等；对于未知尺寸的居中方式都有一定弊端，要么占用了display属性，要么占用了transform导致无法使用相应的动画效果；

CSS

```
.dialog1 {
  /** display属性值被占用 */
  display: flex;
  justify-content: center;
  align-items: center;
}

.dialog2 {
  position: absolute;
  top: 50%;
  left: 50%;
  border: solid;
  /* transform会被animation动画干扰 */
  transform: translate(-50%, -50%);
  animation: tinyUp 0.2s;
}
```

fit-content就很好的填补了未知尺寸居中的空缺；在兼容性方面该关键字在width已得到现代浏览器的全部支持，而在height上firefox还不支持；

CSS

```
.dialog3 {
  position: absolute;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  width: fit-content;
  height: fit-content;
  margin: auto;
  border: solid;
  animation: tinyUp 0.2s;
}
```

min-content

https://codepen.io/dqqbl/pen/MWrbeEN

这个关键字的效果是是content-box区维持“首选最小宽度”，这个“首选最小宽度”对于替换元素取当前元素内容自身的宽度；对于CJK(Chinese/Japanese/Korean)文字，通常指单个文字宽度；对于非CJK文字，通常是单个字符单元(连续的英文字母/数字/标点)宽度;

一个元素最终的首选最小宽度是所有内部子元素最大的那个首选最小宽度值

max-content

让元素尽可能的大，保证图文内容在一行显示，与`white-space:nowrap`类似，但是更高大上

https://codepen.io/dqqbl/pen/MWrbeVg

stretch(fill-available)

stretch指“弹性拉伸”，是最新的规范中定义的关键字，替换之前的fill-available和available，但目前浏览器并不支持，而且没什么实用价值，因为block水平的元素、弹性布局和网格布局中的子项默认都自带 弹性拉伸特性;

1.5 css逻辑属性

css世界是围绕“流”来构建的，但是css属性的定位最初却是基于方向而不是“流”的，这是不合理的；例如，当我们更改文档水平流向的时候，原先通过方向设置的外边距效果就会脱离我们的预期

https://codepen.io/dqqbl/pen/yLpVJGL

css逻辑属性就是用于解决这个问题的，主要就是将原有的方向属性，替换成了 inline/block 和 start/end；

```
margin-left     ↔  margin-inline-start
margin-top      ↔  margin-block-start
margin-right    ↔  margin-inline-end
margin-bottom   ↔  margin-block-end
```

对于包含方向的属性，如margin、padding和border都可以进行类似替换；text-align也支持逻辑属性，但其改变的是值：

- text-align: start
- text-align: end

逻辑它的实用性不强，因为它的使用场景比较有限，而且使用方向值更容易理解一些；比较实用的场景有对话框等对称布局场景，通过改变页面水平流向即可实现 `direction: rtl;`

还有一个比较实用的是 **inset**属性，它对应的是绝对定位的方向属性，如

- `inset-inline-start => left`；
- `inset-block-start => top` ；

但实用的不是它的逻辑，而是它的缩写：

`inset: 0` 等于同与 `left: 0; right:0; top: 0; bottom: 0;`

用来写绝对定位的时候，太香了~

1.6 黏性定位

在过去，黏性定位效果只能通过JavaScript代码实现，常用在导航栏上，如今通过简单的css代码就能实现

无法复制加载中的内容

黏性定位效果底层的渲染规则是相对定位的延伸，它与相对定位的共同点有：

- 元素发生偏移的时候，元素的原始位置是保留的
- 创建了新的绝对定位包含块，黏性定位元素里面如果有绝对定位的子元素，那这个子元素设置left属性、top属性等方向属性时的偏移计算是相对于当前黏性定位元素的
- 支持设置z-index属性值来改变元素的层叠顺序

它的特点有：

- 相对定位偏移计算的容器是父元素， 而黏性定位偏移计算的元素是层级最近的可滚动元素（overflow属性值不是visible的元素) 或浏览器视窗
- 有特殊的偏移计算规则，主要就三个概念，一个是“流盒”，即黏性定位元素最近的可滚动元素的尺寸盒子；二是“黏性约束矩形”，可以简单的理解为黏性定位元素，定位激活之后，它的容器(父元素)，与“流盒”的重叠区，即页面持续滚动，黏性元素还能在页面上“黏”着的滚动长度，滚完了黏性元素就被滚走了

https://codepen.io/dqqbl/pen/PoEbpGL

三是重叠表现不一样，相对定位元素彼此独立，重叠的时候表现为堆叠；但是黏性定位元素在特定布局结构下，元素重叠的时候并不是表现为堆叠，而是会有A黏性定位元素推开B黏性定位元素的视觉表现，即滚动时同一个位置只会有一个黏性元素，新到的会有顶走旧元素的视觉效果

1.7 字体增强

css3全新的通用字体族有

- system-ui：系统UI字体。
- emoji：适用于emoji字符的字体家族。
- math：适用于数学表达式的字体家族。
- fangsong：中文字体中的仿宋字体家族。

其中比较实用的有 `system-ui` 和 `emoji`

`system-ui` 可以调用系统字体，以获得系统上的最佳体验，但还有兼容性问题，暂时还需兜底字体

`emoji` 是表情字体，主流的操作系统都已内置表情字体，我们的孙子系统上已经用起来了

```CSS
.emoji {
  font-family: apple color emoji, segoe ui emoji, noto color emoji, android emoji, emojisymbols,
    emojione mozilla, twemoji mozilla, segoe ui symbol;
}
```

但是这样写一串的方式比较麻烦，可以通过类似这样自定义字体的方式直接引用

```CSS
@font-face {
  font-family: Emoji;
  src: local("Apple Color Emoji"),
  local("Segoe UI Emoji"),
  local("Segoe UI Symbol"),
  local("Noto Color Emoji");
 }
.emoji {
  font-family: Emoji;
}
```

local()函数可以直接调用本地字体，避免发出请求

1.8 background新特性

background-size

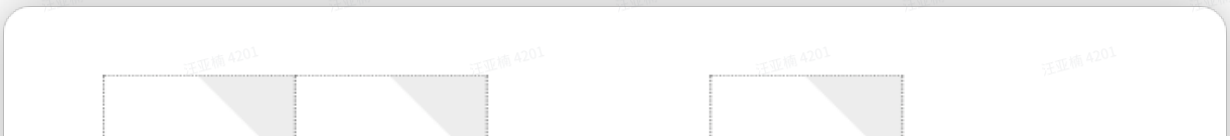
设置背景图的尺寸，除了一般的数值或百分比，还支持几个关键字

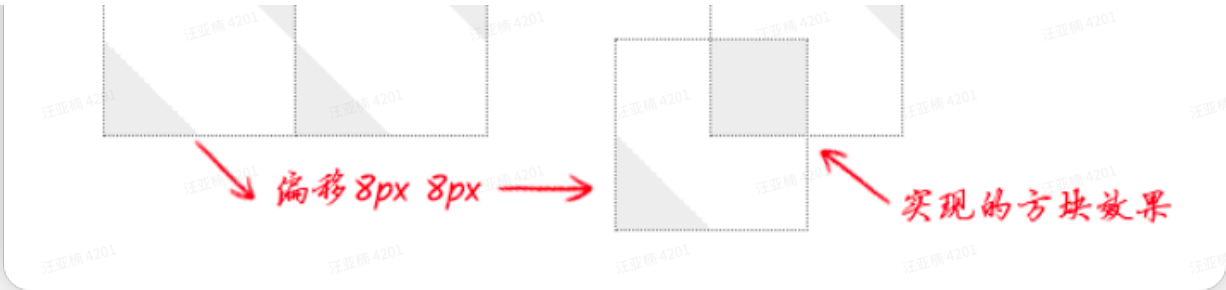
- cover: 覆盖，保留比例，填满空白，裁剪长边
- contain: 包含，保留比例，尽量填满空白，不裁剪
- auto: 根据资源类型和内在比例进行计算

多背景

可以通过逗号分割，设置多个背景图，巧妙的实现一些效果，如通过双背景和修改背景定位实现ps的透明背景效果

https://codepen.io/dqqbl/pen/gOoLmyO



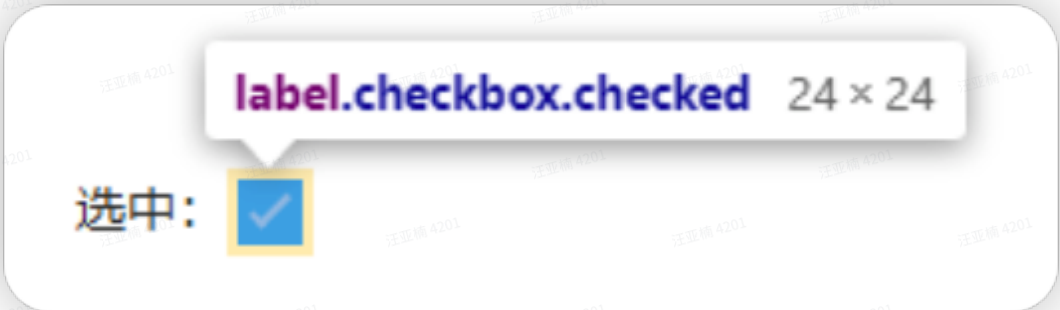


background-clip

用于限制背景的显示区域

```
background-clip: border-box;
background-clip: padding-box;
background-clip: content-box;
background-clip: text;
```

常用的场景在于小按钮通过透明边框扩大可点击区域，同时通过 `background-clip: content-box` 限制背景色的范围



而text关键字，可以实现文字蒙版的效果，实现文字渐变

background-origin

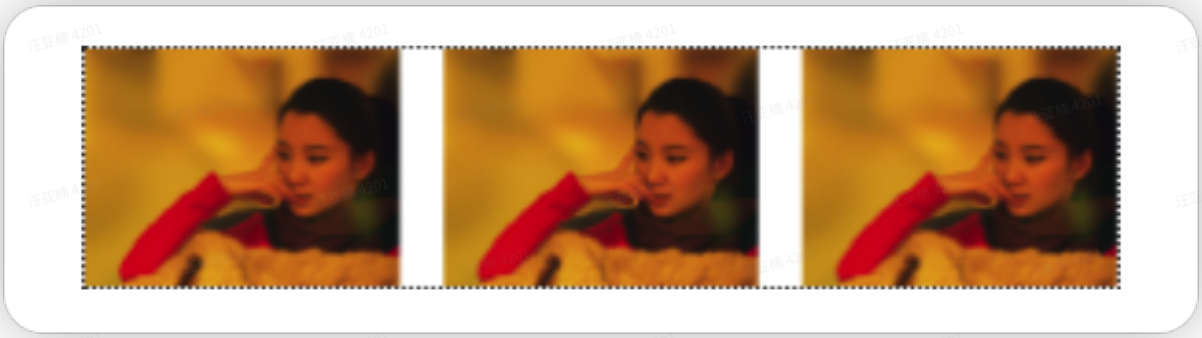
```
background-origin: border-box;
background-origin: padding-box;
background-origin: content-box;
```

这个属性表示背景图像定位的左上角位置哪开始，其默认值是padding-box；之前在做项目时就遇到过这个属性的坑，因为默认值是padding-box，导致border背景色表现和内容区不一致，可以将值设为border-box 解决

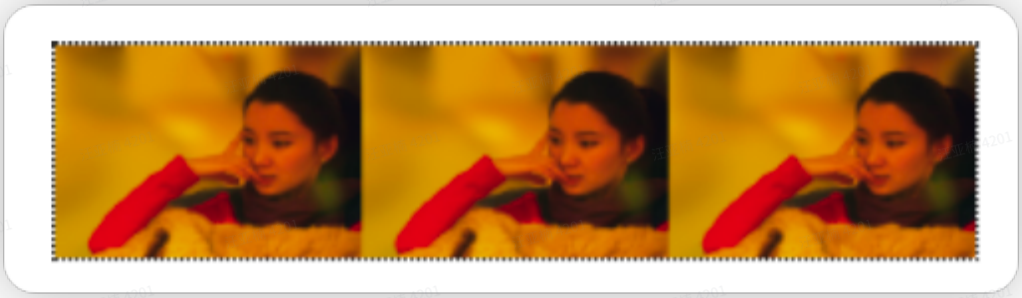
https://codepen.io/dqqbl/pen/ExoNmmO

space / round

这是background-repeat新增的两种平铺模式，space可以实现类似grid的平铺效果，让背景图尽可能的重复，不进行裁剪填满背景，剩余空间用空白间隙均匀分布；



round与space类似，但会通过平均拉伸填满空白



1.9 color属性与颜色设置

transparent关键字

transparent在css3之前只是一个特殊的关键字，到css3<color>数据类型开始支持Alpha通道，transparent关键字才被重新定义为真正的颜色；通过transparent我们可以实现我们视觉认知中的渐

变

https://codepen.io/dqqbl/pen/JjMEqdv

但这其实是浏览器针对transparent实现了特殊的算法，如果仅通过数值变化实现渐变，则会出现一种脏脏的感觉；在svg和canvas中的渐变就是这种效果；

currentColor关键字

它表示当前元素 (或伪元素)所使用的color属性的计算值，可以用在svg的颜色填充中

HSL颜色

HSL颜色是由色调(Hue)、饱和度(Saturation)和亮度 (Lightness)这3个元素组成的，如hsl(45,100%,50%)

色调值的大小是任意的，因为CSS世界中与色调相关的数值都是角度值，其单位是deg，所以无论设置什么值，最终都会在0~360deg这个范围中解析，例如－120deg=240deg、480deg=120deg；

饱和度和亮度的值也是任意的，最终解析的数值范围均在0%~ 100%，数值后面的百分号是必须的；下面是色值hsl(0,100%,50%)到 hsl(360,100%,50%)的色值带，可以看出来HSL颜色色调从0~360deg大致按照红、橙、黄、绿、青、蓝、紫、红的颜色顺序分布

#RRGGBBAA颜色

#RRGGBBAA颜色的最后两位表示透明度，如`background:#FF000080`;本来是很方便简写透明度的方式，可惜最后两位也是十六进制的，需要自行转换对应的透明度

新语法

现代浏览器支持全新的空格加斜杠语法，稍微方便了一点

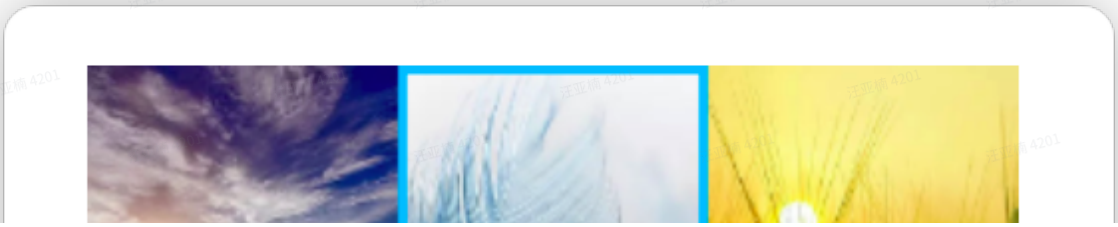
```
CSS
rgb(255 0 153 / 1)
rgb(255 0 153 / 100%)
rgba(51 170 51 / 0.4)
rgba(51 170 51 / 40%)
hsl(270 60% 50% / .15)
hsl(270 60% 50% / 15%)
```

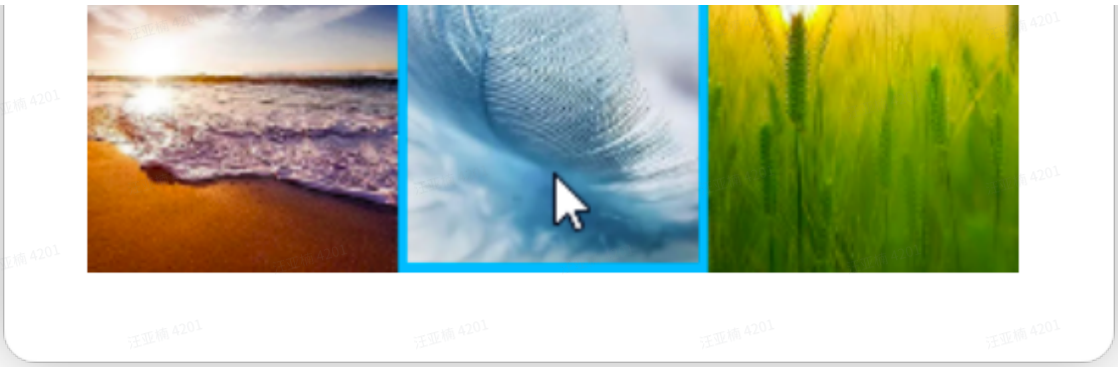
1.10 其它

outline-offset

outline-offset属性用于改变outline属性设置的轮廓的偏移位置。通常我们会取负值；可用于tab键选取的展示效果

```
CSS
img {
  outline-offset: -3px;
}
```





cursor

cursor新增了 zoom-in / zoom-out （放大缩小）和 grab / grabbing （摘取）的效果

https://codepen.io/dqqbl/pen/YzYNbLo